

Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas

Alumno

Galicia Cobaxin Daniel

Profesor

Noe Sierra Romero

Unidad de aprendizaje

Aplicaciones distribuidas

Practica:

MongoDB

1. Objetivo

El objetivo de esta práctica es establecer una conexión exitosa entre una aplicación Node.js y un clúster de MongoDB Atlas (servicio en la nube), así como ejecutar consultas básicas sobre las colecciones de la base de datos de ejemplo sample_mflix, que contiene información sobre películas, comentarios y embeddings vectoriales.

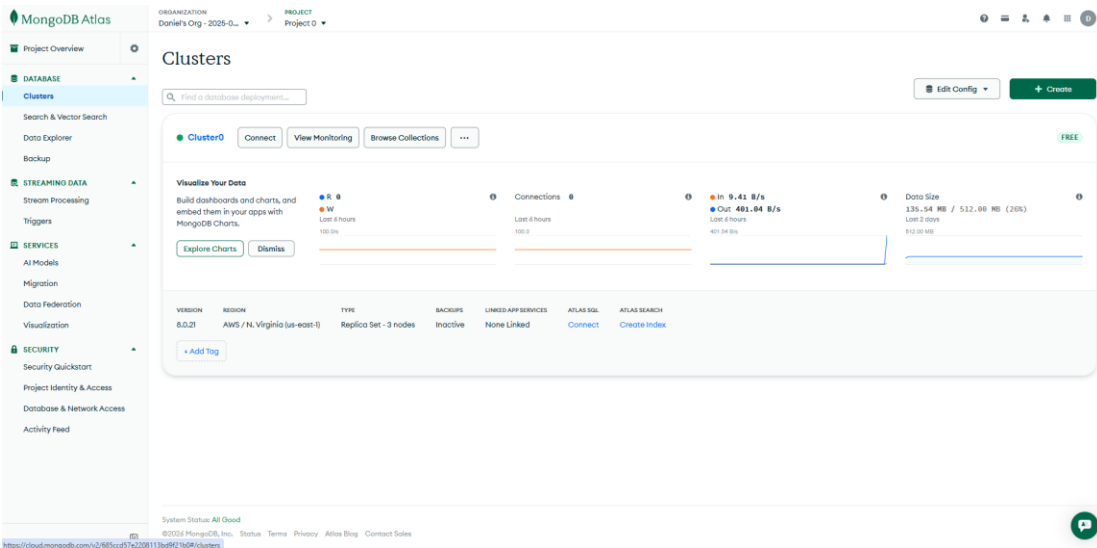
2. Herramientas y tecnologías utilizadas

Herramienta	Versión / Plan	Descripción
Node.js	v24.13.0	Entorno de ejecución de JavaScript del lado del servidor
Driver mongodb	v6 (npm)	Paquete oficial para conectar Node.js con MongoDB
MongoDB Atlas	Free Tier (M0)	Servicio en la nube de MongoDB con 512 MB gratuitos
Base de datos	sample_mflix	Dataset de ejemplo con películas, comentarios y vectores
Sistema operativo	Windows 11	Entorno local de desarrollo

3. Configuración del entorno

3.1 Creación del clúster en MongoDB Atlas

Se creó una cuenta gratuita en MongoDB Atlas (cloud.mongodb.com) y se configuró un clúster M0 (Free Tier) con las siguientes características:



3.2 Configuración de seguridad

Para permitir la conexión desde el entorno local se realizaron los siguientes pasos en el panel de MongoDB Atlas:

- Se creó un usuario de base de datos con permisos de Atlas Admin.
- Se agregaron las IPs autorizadas en la lista de acceso de red (Network Access).
- Se utilizó el DNS de Google (8.8.8.8) como servidor alternativo debido a restricciones del ISP local que bloqueaba la resolución de registros SRV.

3.3 Inicialización del proyecto Node.js

En la terminal se ejecutaron los siguientes comandos para inicializar el proyecto e instalar las dependencias necesarias:

```
mkdir practica-mongodb
cd practica-mongodb
npm init -y
npm install mongodb
```

4. Código fuente

A continuación se presenta el archivo index.js desarrollado para la práctica. El programa está estructurado en funciones asíncronas independientes, cada una con una responsabilidad específica:

```
const { MongoClient } = require('mongodb');

const uri = "mongodb+srv://<usuario>:<password>@cluster0.xxxxx.mongodb.net/";
const client = new MongoClient(uri);

// Lista todas las bases de datos disponibles en el clúster
async function listDatabases(client) {
  const databasesList = await client.db().admin().listDatabases();
  console.log('Databases:');
  databasesList.databases.forEach(db => console.log(` - ${db.name}`));
}

// Consulta las primeras 2 películas de la colección movies
async function findAllData(client) {
  const cursor = await
  client.db('sample_mflix').collection('movies').find({}).limit(2);
```

```
const results = await cursor.toArray();
console.log('Title:', results[0]['title']);
console.log(JSON.stringify(results, null, 2));
}

// Consulta los primeros 5 comentarios de la colección comments
async function findComments(client) {
  const cursor = await
client.db('sample_mflix').collection('comments').find({}).limit(5);
  const results = await cursor.toArray();
  console.log(JSON.stringify(results, null, 2));
}

// Consulta las primeras 5 películas con embeddings vectoriales
async function findEmbeddedMovies(client) {
  const cursor = await
client.db('sample_mflix').collection('embedded_movies').find({}).limit(5);
  const results = await cursor.toArray();
  console.log(JSON.stringify(results, null, 2));
}

async function main() {
  try {
    await client.connect();
    await findComments(client);
    await findEmbeddedMovies(client);
  } catch (e) {
    console.error(e);
  } finally {
    await client.close();
  }
}

main().catch(console.error);
```

5. Estructura de las colecciones consultadas

5.1 Colección: movies

Contiene información detallada de películas. A continuación, se describe la estructura de sus campos principales:

Campo	Tipo de dato	Descripción
_id	ObjectId	Identificador único generado automáticamente por MongoDB
title	String	Título de la película
year	Integer	Año de estreno de la película
plot	String	Sinopsis o descripción breve de la trama
genres	Array<String>	Lista de géneros cinematográficos
cast	Array<String>	Lista de actores que participan en la película
imdb.rating	Double	Calificación de la película en la plataforma IMDb
runtime	Integer	Duración total de la película en minutos

5.2 Colección: comments

Almacena los comentarios realizados por usuarios sobre las películas. Cada documento contiene una referencia a la película correspondiente mediante el campo `movie_id`.

Campo	Tipo de dato	Descripción
_id	ObjectId	Identificador único del comentario
name	String	Nombre del usuario que realizó el comentario
email	String	Correo electrónico del usuario
movie_id	ObjectId	Referencia al <code>_id</code> de la película comentada
text	String	Contenido del comentario escrito por el usuario
date	Date	Fecha y hora en que se publicó el comentario

5.3 Colección: embedded_movies

Es una versión extendida de la colección `movies` que incluye el campo `plot_embedding`. Este campo contiene un vector numérico de aproximadamente 1,500 dimensiones generado por un modelo de inteligencia artificial a partir de la sinopsis de cada película. MongoDB utiliza estos vectores para realizar búsquedas semánticas, permitiendo encontrar películas con significados similares, aunque no compartan las mismas palabras clave.

```
{
  "_id": "5a9427648b0beeb6957a38",
  "name": "Yara Greyjoy",
  "email": "gemma_whelan@gameofthron.es",
  "movie_id": "573a1390f29313caabcd587d",
  "text": "Nobis incidunt ea tempore cupiditate sint. Itaque beatae hic ut quis.",
  "date": "2012-11-26T11:00:57.000Z"
},
{
  "_id": "5a9427648b0beeb6957b18",
  "name": "Doreah",
  "email": "roxanne_mckee@gameofthron.es",
  "movie_id": "573a1391f29313caabcd7915",
  "text": "Porro explicabo cumque vitae sint quia quis. Consequuntur voluptatibus optio maxime ratione temporibus incidunt minus architecto.",
  "date": "1983-09-13T03:38:23.000Z"
},
{
  "_id": "5a9427648b0beeb6957b86",
  "name": "Sansa Stark",
  "email": "sophie_turner@gameofthron.es",
  "movie_id": "573a1391f29313caabcd8268",
  "text": "Aliquam tempora suscipit vero id. Natus nesciunt molestiae odio. Vel molestiae incidunt laborum fuga. Excepturi labore molestias dolor.",
  "date": "1996-10-07T18:10:43.000Z"
},
{
  "_id": "5a9427648b0beeb6957d89",
  "name": "Missandei",
  "email": "nathalie_emmanuel@gameofthron.es",
  "movie_id": "573a1392f29313caabcd9b68",
  "text": "Nulla esse incidunt quisquam praesentium. Sapiente voluptates tempore numquam fugiat. Minima odio in dignissimos officiis. Quod ex libero facer",
  "date": "2013-10-03T22:52:21.000Z"
},
{
  "_id": "5a9427648b0beeb6957efb",
  "name": "Osha",
  "email": "natalia_tena@gameofthron.es",
  "movie_id": "573a1392f29313caabcd9b68",
  "text": "Voluptatem quae pariatur ut rem laudantium. Quasi sed iste ipsum laborum. Quia placeat qui vero adipisci.",
  "date": "1973-02-23T15:15:58.000Z"
}
```

```
,
  "lastUpdated": "2015-07-22T19:23:50.000Z",
  "rotten": 0,
  "fresh": 9
},
"plot_embedding": [
  -0.005927917,
  -0.033394486,
  0.0015323418,
  -0.037410278,
  -0.020594146,
  0.027185857,
  -0.007820227,
  -0.0030944059,
  0.0093459645,
  -0.016234897,
  0.01850699,
  0.0038110397,
  0.03463621,
  -0.018718349,
  0.01768798,
  -0.011340651,
  0.038942616,
  -0.014253421,
  0.02188871,
  0.007523006,
  0.0024966605,
  -0.0051980736,
  0.007840042,
  0.004504557,
  0.014517617,
  -0.01776724,
  -0.010349913,
  -0.023288954,
  0.041056193,
  -0.009035532,
  0.022707721,
  -0.013249473,
  0.011664292,
  0.01049522,
```

8. Conclusiones

La práctica permitió comprender y aplicar los siguientes conceptos fundamentales de MongoDB y las bases de datos NoSQL:

- MongoDB almacena los datos en documentos BSON (similar a JSON), lo que permite estructuras flexibles y anidadas, a diferencia de las tablas rígidas de las bases de datos relacionales.
- MongoDB Atlas facilita el despliegue de bases de datos en la nube sin necesidad de instalar software localmente, ofreciendo un plan gratuito adecuado para prácticas y proyectos pequeños.
- El driver oficial de MongoDB para Node.js permite interactuar con la base de datos de forma asíncrona utilizando `async/await`, lo que resulta en código limpio y fácil de mantener.
- Los métodos `find()`, `limit()` y `toArray()` son las operaciones básicas de consulta (lectura) en MongoDB, equivalentes al `SELECT` de SQL.
- La colección `embedded_movies` introduce el concepto de vectores de embeddings, una tecnología de inteligencia artificial que permite realizar búsquedas semánticas avanzadas sobre texto.